Internet Information Service (IIS)

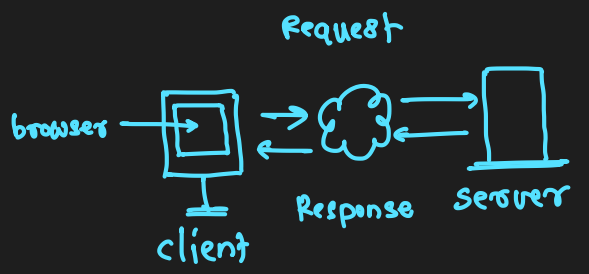
Web Server

Used to host websites

Introduction

- A **web server** is a software application (and the underlying hardware it runs on) that stores, processes, and delivers web content (web pages, files, images, videos, APIs) to clients over the internet or an intranet using the **HTTP/HTTPS** protocol
  ↳ resources
- It operates on a **client-server model**:
  - Client (usually a web browser) sends a request, and the server responds with the requested resource or an error message
- **Popular web servers**
  Redhat → HTTPD , Debian → apache2
  - Apache HTTP Server: Open-source, modular, .htaccess, widely used
  - Nginx: High performance, event-driven, reverse proxy, load balancing
  - Microsoft IIS: Windows-based, integrates with ASP.NET
  - LiteSpeed: Fast, Apache-compatible, low resource usage
  - Apache Tomcat: Java Servlet/JSP container → application server
  - Caddy: Automatic HTTPS, modern, simple config
  - Node.js (Express): JavaScript runtime used as a lightweight server

Web architecture



Functionalities

↳ hosting a static web site

- **Serving static content:** Delivers HTML, CSS, JS, images, PDFs directly from disk
- **Serving dynamic content:** Executes scripts (PHP, Python, Node.js) and returns generated output ↳ hosting dynamic web site
- **Request handling:** Parses HTTP requests, methods (GET, POST, PUT, DELETE)
- **Response generation:** Sends status codes (200, 404, 500) + headers + body
- **Logging:** Records access logs and error logs
- **Security:** TLS/SSL encryption, authentication, access control ⇒ https
- **Virtual hosting:** Hosts multiple domains on one server
- **Load balancing / proxying:** Distributes traffic across backend servers

How a Web Server Works

- **DNS Resolution** – Browser resolves domain name → IP address.
- **TCP Connection** – Client establishes a connection (usually port 80 for HTTP, 443 for HTTPS).
- **HTTP Request** – Browser sends a request:
  - GET /index.html HTTP/1.1
  - Host: www.example.com
- **Server Processing** – Server locates resource, runs scripts if needed.
- **HTTP Response** – Server returns:
  - HTTP/1.1 200 OK
  - Content-Type: text/html
- **Rendering** – Browser renders the page.
- **Connection Close/Keep-alive** – Depending on headers.

Types of Web Servers

Static website = website developed in html + css + JS

Dynamic website = website developed in server sided programming languages
= eg. Java, python, javascript

- **Static Web Server**
  - Serves pre-built files as-is. Fast and simple. Example: serving a plain HTML site
- **Dynamic Web Server**
  - Combines a web server with an application server to generate content on the fly (e.g., database-driven sites)
- **Proxy / Reverse Proxy Server**
  - **Forward proxy** – Acts on behalf of clients
  - **Reverse proxy** – Acts on behalf of servers (e.g., Nginx, HAProxy) for load balancing, caching, SSL termination

Core Components



- **Listener/Socket** – Accepts incoming connections
- **Request Parser** – Interprets HTTP request
- **Handler/Module** – Processes request (static, CGI, proxy, etc.)
- **Thread/Process Pool** – Concurrency model (threaded, event-driven, process-based)
- **Document Root** – Base directory for files
- **Configuration Files** – httpd.conf, nginx.conf, .htaccess
- **MIME Types** – Maps file extensions to content types
- **Cache** – Stores frequent responses for speed


IIS

IIS



- **Internet Information Services (IIS)** is Microsoft's built-in web server role for Windows Server and Windows Client operating systems
- It provides a secure, manageable, and scalable platform for hosting websites, web applications, and services. IIS 10.0 is the current version included with supported Windows Server releases, including Windows Server 2025
- IIS is **not installed by default** on a fresh Windows Server installation., it must be added via Server Manager or PowerShell
- **Installation**
  - IIS is installed as a Web Server (IIS) role via
    - Server Manager → Add Roles and Features → Web Server (IIS)
    - PowerShell
      - Install-WindowsFeature -Name Web-Server
  - Default Installation
    - Creates a "Default Web Site" listening on HTTP port 80, serving content from %SystemDrive%\inetpub\wwwroot

Core Architecture

Application Pool → Website



- **Web Sites**
  - Top-level containers representing individual websites
  - Each site is identified by a unique combination of IP address, port, and host name (binding)
- **Applications**
  - A subset of a site that runs as a distinct unit
  - Applications can have their own application pool, configuration settings, and even a separate .NET runtime
- **Virtual Directories**
  - Logical directory names that map to physical folders on disk (can be outside the site root)
  - They appear as normal subfolders to users
- **Application Pools**
  - Worker process environments where sites/applications run
  - Each pool runs in its own sandboxed process (w3wp.exe), providing isolation

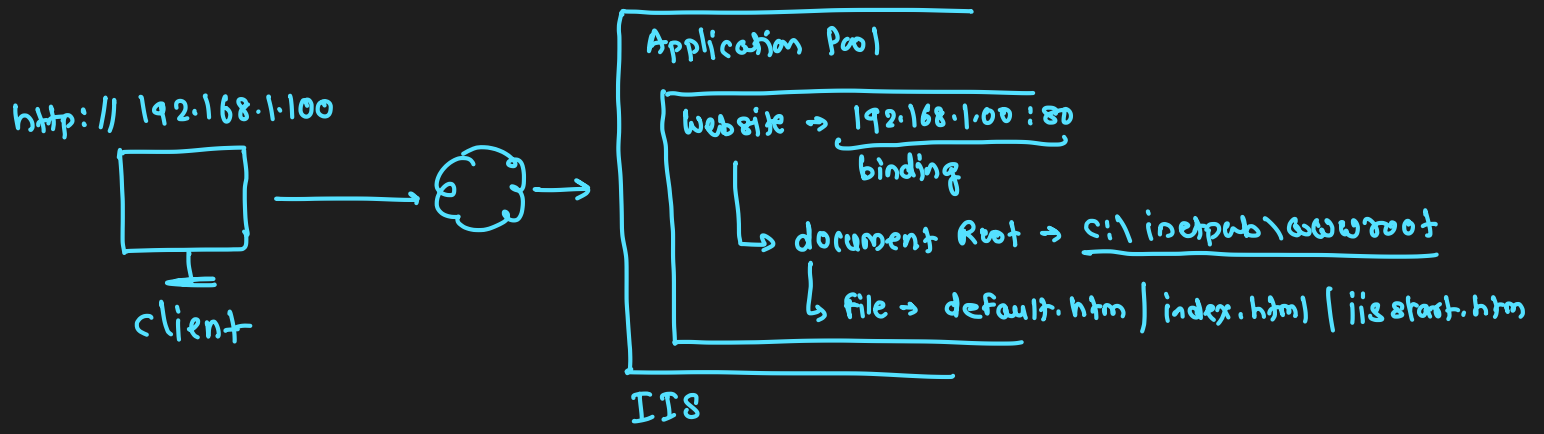
WWW - Worker Process

- **Modules**
  - Components that handle specific request-processing tasks (authentication, compression, URL rewriting).
  - Modules can be added or removed as needed

Request Processing Flow

- **Site Identification**
  - Incoming request matched to a site via IP, port, and Host header against site bindings
- **Path Resolution**
  - URL path examined to determine if it maps to an application or virtual directory with distinct settings
- **Execution**
  - Request executed using the assigned application pool, which loads necessary modules for authentication, authorization, content serving, or code execution

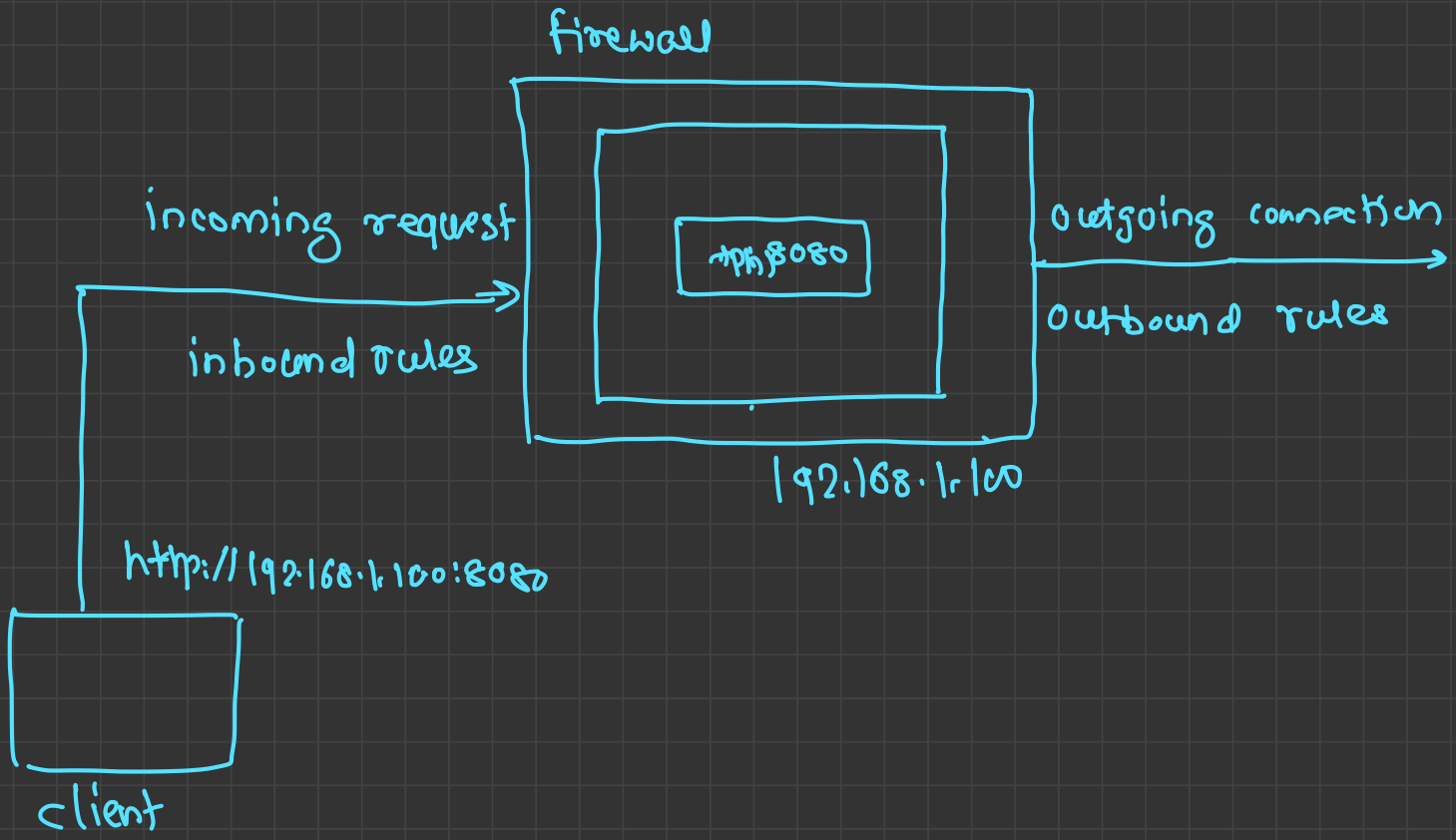
Binding ⇒ mapping IP address & port



Creating and Configuring Websites



- Open **IIS Manager** (inetmgr.exe)
- In Connections pane, expand server node → right-click **Sites** → **Add Website**
- Configure
  - **Site name:** Descriptive name
  - **Application pool:** Default creates new pool with same name
  - **Physical path:** Content directory (e.g., C:\inetpub\contoso)
  - **Binding type:** HTTP or HTTPS
  - **IP address:** Usually "All Unassigned"
  - **Port:** 80 (HTTP) or 443 (HTTPS)
  - **Host name:** FQDN (required when multiple sites share the same port/IP)


firewall
incoming request
inbound rules
nginx:8080
outgoing connection
outbound rules
192.168.1.100
http://192.168.1.100:8080
client